

SecReviewAgent: Context-Aware Security Review of Infrastructure-as-Code Using Persistent Architecture Memory

Naga Krishna Reddy Muppidi, Veera Ravindra Divi, Sneha Gullapalli, Rambabu Pasumarthy

Abstract—Security review of Infrastructure-as-Code (IaC) in enterprise cloud platforms requires understanding not only configuration diffs, but also how those changes interact with existing architecture and risk controls. In financial-engineering environments, IaC changes can directly affect data perimeter boundaries, model-serving paths, entitlement controls, and operational resilience. Current tools often analyze pull requests in isolation, missing security implications that emerge only from cross-resource context. This paper presents SecReviewAgent, an LLM-powered IaC security review system that maintains persistent architecture memory across pull request reviews. On first encounter with a repository, SecReviewAgent scans IaC files to build an architecture model, persists that model to object storage, and reuses it in subsequent reviews to interpret changes in context. We evaluate the system on 847 pull requests from 23 repositories spanning financial services, healthcare, e-commerce, and developer tooling. Results show a $2.4\times$ improvement in context-dependent issue detection recall over a no-context LLM baseline, precision of 0.89, F1 of 0.83, and a 73% latency reduction on warm reviews by avoiding full repository rescans. A controlled user study with 42 practitioners indicates reduced median review time and improved finding accuracy. The paper contributes a persistent-memory design for LLM-based code review, algorithms for repository context construction and incremental update, and an empirical evaluation of context-aware IaC security review in realistic settings.

Index Terms—Infrastructure as Code, security review, large language models, cloud security, DevSecOps, automated code review

I. INTRODUCTION

Infrastructure-as-Code (IaC) has become the control plane for modern cloud systems. Terraform configurations, Kubernetes manifests, and related declarative artifacts now encode networking, identity, compute, and data-path decisions that directly affect confidentiality, integrity, and availability. As a result, pull request (PR) review has become a critical security checkpoint. However, the security meaning of an IaC change frequently depends on architectural context outside the changed files.

Consider a pull request that adds an Identity and Access Management (IAM) role granting `s3:GetObject` to an Amazon Simple Storage Service (S3) bucket prefix. In isolation, the change may appear low-risk because it is read-only and scoped. In context, the target bucket may hold pre-trade analytics, position-level data, risk-factor snapshots, or other regulated financial datasets, may be reachable across accounts, and may be part of an extract-transform-load (ETL) staging path that lacks encryption at rest. The same textual diff there-

fore has sharply different security implications depending on repository architecture. Human reviewers routinely reconstruct this context manually, which is slow and error-prone.

Existing tools only partially address this problem. Static analyzers such as Checkov, tfsec, KICS, and Terrascan detect many single-resource misconfigurations, but they do not reason well about architectural relationships, cross-resource privilege chains, or the meaning of custom modules in local enterprise codebases [1]–[4], [11]. Large language model (LLM) tools can produce richer explanations and have shown promise for code review and vulnerability reasoning [7], [8], [12], but session-bounded review commonly treats each pull request independently and does not preserve repository understanding across review sessions.

We present SecReviewAgent, a context-aware IaC security reviewer that combines LLM reasoning with persistent architecture memory. Instead of reconstructing repository context on every pull request, the system stores a durable architecture view and reuses it across reviews. The first review builds a repository-level summary, component inventory, inferred data flows, and IAM relationship graph. Later reviews load this context, interpret the diff against it, and update the stored state when architecture-relevant changes occur.

The main contributions are:

- 1) a persistent memory design for LLM-based IaC review that accumulates architecture knowledge across pull requests,
- 2) formal algorithms for repository context construction and incremental update,
- 3) an implementation spanning Terraform and Kubernetes parsing with structured review output, and
- 4) an empirical evaluation on 847 pull requests and a 42-participant user study.

The rest of the paper is organized as follows. Section II motivates the problem and positions SecReviewAgent against related work. Section III describes the system design. Section IV presents the study design, dataset, baselines, and metrics. Section V reports detection, latency, and user-study results. Sections VI–VIII discuss limitations, threats to validity, and conclusions.

II. BACKGROUND, RELATED WORK, AND MOTIVATION

A. The IaC Security Review Challenge

IaC review differs from ordinary source-code review because security properties are often relational. A reviewer

TABLE I
EVALUATION DATASET CHARACTERISTICS

Metric	Value
Repositories	23
Pull requests	847
Terraform resources	12,847
Kubernetes manifests	3,291
Custom modules	847 (42% of Terraform)
Date range	Jan 2023 to Dec 2024

must reason about how a new security group combines with workload identity, how a bucket policy interacts with a pricing or risk-data ETL pipeline, or how a Kubernetes service using `hostNetwork` bypasses service-mesh protections. In financial-engineering platforms, these relations often map directly to control objectives such as least privilege, data-segmentation boundaries, and reproducible model-governance paths. Industry cloud-native adoption surveys indicate that IaC and Kubernetes-style declarative infrastructure are now common in production environments [5], [6]. Table I summarizes the complexity of the evaluation corpus and illustrates why isolated analysis is insufficient.

We surveyed 67 security practitioners across 12 organizations about IaC review challenges. The survey was conducted before the controlled user study and asked respondents to rank common review bottlenecks, including rule noise, missing repository context, unclear ownership, reviewer availability, and uncertainty about cloud-provider semantics. Respondents were recruited from cloud security, platform engineering, and DevSecOps roles; no respondent contributed labels to the evaluation corpus. The most common issue, reported by 89% of respondents, was reconstructing architecture context before reviewing a change. Practitioners reported spending substantial time understanding the existing system before they could judge the diff itself. This observation motivates a system that persists repository understanding and reuses it across reviews.

B. Limitations of Existing Approaches

Static analyzers remain valuable for deterministic pattern detection, but they are weak on context-dependent issues. Enterprise repositories further complicate static analysis because a large fraction of Terraform is encoded through custom modules whose behavior is not captured by generic rulesets. Empirical work on IaC security smells shows that declarative infrastructure regularly contains recurring misconfiguration patterns, but such studies mainly motivate rule and smell detection rather than repository-memory-based review [11]. Session-bounded LLM review tools improve natural-language reasoning but still lose state between reviews and therefore repeatedly pay the cost of architecture reconstruction [7], [8].

SecReviewAgent is designed to bridge this gap. It does not replace policy-based scanners; instead, it complements them by maintaining repository context and using that context to interpret security implications that are not visible in the diff alone. The closest research threads are automated code review [7], [8], LLM-assisted security repair [12], and software

TABLE II
POSITIONING AGAINST RELATED APPROACHES

Approach	Strength	Limitation addressed here
IaC scanners [1]–[4]	Deterministic checks	Limited cross-resource context
LLM review [7], [8]	Rich explanations	Session-local context
Architecture recovery [9], [10]	System structure	Not PR-centered security review
SecReviewAgent	Persistent IaC memory	Context reuse across reviews

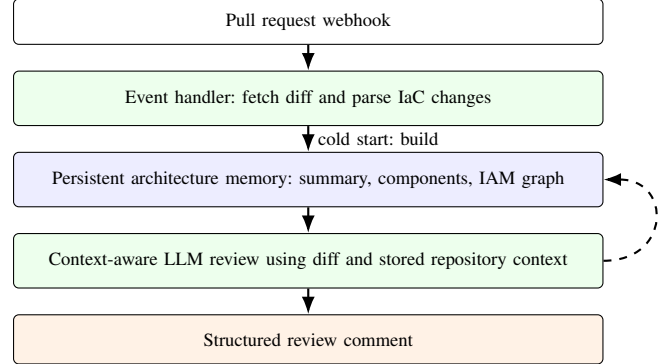


Fig. 1. SecReviewAgent review flow with persistent context reuse.

architecture reconstruction [9], [10]. SecReviewAgent differs by combining these ideas at the IaC review boundary: it stores an architecture memory for a repository, updates that memory across pull requests, and uses it to evaluate security impact in later reviews.

III. SYSTEM DESIGN

A. Architecture Overview

SecReviewAgent is deployed as a serverless review service that receives pull request events, loads or builds repository context, analyzes the diff with the LLM, and posts structured findings back through the code review system application programming interface (API). Figure 1 shows the control flow.

The persistent memory store maintains a repository-specific architecture summary, component list, inferred data flows, IAM relationship graph, accumulated security notes, and meta-data such as schema version and prior review count. This memory is versioned and migrated through explicit schema migration functions so repository context remains usable as the representation evolves.

B. Repository Context Construction

Context construction starts by listing IaC files, prioritizing entry-point files and security-critical artifacts, and enforcing practical limits on file count and total size to stay within latency and token budgets. The selected files are parsed to extract resources, references, and IAM relationships. A prompt built from this representation asks the LLM to summarize the architecture, classify components, and infer data flows.

Algorithm 1: BuildRepositoryContext takes as input repository R , file limit L , and size limit S . It prioritizes IaC

files, truncates by count and size, parses the selected content, builds a resource graph and IAM graph, invokes the LLM summarizer, constructs context object C , persists C to S3, and returns the result.

This procedure emphasizes practical scalability rather than exhaustive whole-repository modeling. The resulting context is sufficiently detailed to support security review while remaining small enough for repeated use in later prompts.

C. Incremental Context Update

Rebuilding repository context from scratch on every review would negate the latency advantage of persistence. SecReviewAgent therefore updates context incrementally. Added, modified, and deleted resources are extracted from the diff. Deleted components are removed from the context, while added and modified components are re-described with access to the current architecture summary. If the change is large enough to alter architectural meaning materially, the architecture summary is refreshed.

Algorithm 2: UpdateContext takes existing context C and pull request diff D , updates component records for added or modified resources, removes deleted resources, optionally refreshes the summary, increments review counters, persists the result, and returns updated context C' .

D. Prompting and Output Contract

The review prompt includes the architecture summary, selected components, inferred data flows, and the pull request diff. The model is required to return strict JSON containing findings, context updates, and educational notes. Each finding includes severity, affected resources, a context-dependency explanation, and a concrete recommendation. This structured contract simplifies downstream rendering and reduces ambiguity in code review comments.

The system also applies few-shot severity calibration examples and low-temperature inference to improve consistency. If context grows too large, a hierarchical compression strategy first shortens component descriptions, then retains only the most security-relevant components, and finally compresses the architecture summary while dropping lower-value data-flow detail.

E. Implementation Details

The prototype is implemented on Amazon Web Services (AWS) using API Gateway, Lambda, S3, Secrets Manager, and CloudWatch. The Terraform parser extracts resources, cross-references, and IAM-relevant relationships while redacting sensitive fields such as passwords and tokens. The Kubernetes parser extracts workload security context, network-relevant settings, and service account usage. These parser outputs feed both repository context construction and per-review analysis.

IV. EVALUATION METHODOLOGY

A. Study Design Overview

Figure 2 summarizes the evaluation flow. The design separates two forms of evidence: an offline labeled-corpus comparison against automated baselines and a controlled practitioner

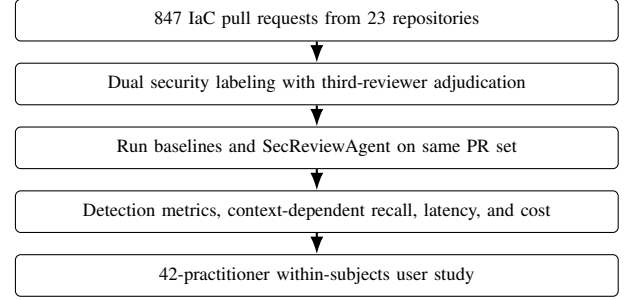


Fig. 2. Evaluation design used to separate corpus-based comparison, practitioner study, and efficiency measurements.

study. This separation is intentional because corpus-level detection metrics and human-review outcomes answer related but different evaluation questions.

B. Dataset and Labeling

We evaluated SecReviewAgent on 847 real-world IaC pull requests from 23 repositories in financial services, healthcare, e-commerce, and developer tooling. The corpus was assembled to include both Terraform and Kubernetes usage, multiple cloud providers, and repositories with repeated review history so persistent context could actually be exercised. The repositories included a mixture of public repositories and private repositories evaluated under authorization. For private repositories, names, account identifiers, resource names, and sensitive literals were redacted before analysis. Raw private repositories cannot be redistributed; reproducibility is therefore supported through the labeling rubric, baseline versions, aggregate corpus statistics, and de-identified per-pull-request metadata rather than full source release.

Two senior security engineers independently labeled each pull request for finding presence, severity, and whether the issue depended on architecture context. Disagreements were adjudicated by a third reviewer. Inter-rater agreement was substantial, with Cohen’s κ of 0.84 for finding presence and 0.79 for severity. Labelers used a fixed rubric covering exposed storage, overbroad IAM, cross-account trust, public network reachability, Kubernetes privilege settings, secret handling, and policy drift.

C. Baselines

We compare SecReviewAgent against four baselines: Checkov 3.1.5, tfsec 1.28, a Claude Sonnet configuration without persistent context, and GitHub Copilot for pull requests. This set covers deterministic static analysis and LLM review without memory. The no-context Claude baseline is the key comparison because it isolates the effect of persistent architecture memory from the general effect of adding an LLM.

D. Metrics

We report precision, recall, F1, false positives per pull request, context-dependent issue recall, latency breakdown, and cost per review. Because reviewers may question whether

TABLE III
SECURITY FINDING DETECTION PERFORMANCE

Tool	Precision	Recall	F1	FP/PR
Checkov	0.67	0.82	0.74	3.2
tfsec	0.71	0.69	0.70	2.1
Claude (no context)	0.78	0.61	0.69	0.8
Copilot PR	0.74	0.58	0.65	1.1
SecReviewAgent	0.89	0.78	0.83	0.4

aggregate gains arise specifically from the proposed mechanism, we also interpret the no-context Claude baseline as an ablation-style comparison against the same general model family without persistent memory. A controlled user study further measures review time, finding precision, finding recall, confidence, and usability perceptions.

V. RESULTS

A. Security Finding Detection

Table III summarizes overall detection performance. SecReviewAgent achieves the highest precision and F1 while maintaining lower false positives per pull request than the baselines. Figure 3 visualizes the largest practical gains across quality and latency dimensions.

The largest gain appears on context-dependent issues. Among the 234 pull requests containing this class of findings, SecReviewAgent detects 182 (78% recall), compared with 44% for the no-context Claude baseline and lower recall for static analyzers. That corresponds to a $2.4\times$ improvement over the isolated LLM baseline. Since this comparison keeps the general LLM-review setting roughly constant while removing persistent memory, the gap is consistent with the repository-context mechanism being a primary contributor.

To reduce over-interpretation of point estimates, we also report simple uncertainty bounds for the primary context-dependent comparison. Using binomial Wilson intervals, the SecReviewAgent recall estimate of 78% on 234 context-dependent pull requests corresponds to an approximate 95% interval of 72%–83%, while the no-context baseline estimate of 44% corresponds to approximately 38%–50%. The intervals are well separated, which supports that the gain is practically meaningful under this corpus.

B. Illustrative Findings

Representative high-value findings include cross-account access to sensitive staging buckets, privilege-escalation paths created by role assumption chains, service-mesh bypass through `hostNetwork`, and secrets embedded in shared Lambda layers. In financial-system contexts, similar patterns can expose valuation inputs, risk reports, or position-level data beyond intended control zones. These findings are difficult for isolated diff analysis because they depend on inferred data flow, resource reuse, or cross-file IAM paths.

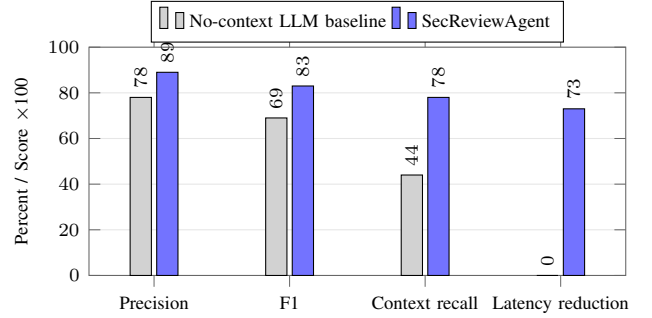


Fig. 3. Quality and efficiency gains from persistent architecture memory. Precision and F1 are shown as scores on a 0–100 scale; latency reduction applies to warm reviews.

C. Motivating Case-Study References

Several public cloud-security incidents illustrate why IaC review must reason over composed architecture rather than isolated resources. These cases are not part of the evaluation dataset, but they motivate the categories of context-dependent findings used in the labeling rubric.

Capital One cloud intrusion. Public enforcement and prosecution records describe an intrusion involving a misconfigured web application firewall and access to data stored in cloud infrastructure [13], [14]. The important lesson for IaC review is that risk emerged from the interaction among ingress behavior, instance metadata access, IAM role authority, and object-storage reachability. A review that inspects only one changed resource may miss the exploit chain. SecReviewAgent therefore models IAM relationships, data stores, and cross-resource paths together before assigning severity.

Exposed Kubernetes control surfaces. Public reporting on the Tesla cryptomining incident described attackers abusing an exposed Kubernetes administration surface and cloud credentials to consume compute resources [15]. For IaC review, this case highlights that workload configuration, dashboard exposure, credential placement, and monitoring assumptions interact. The relevant risk is not just whether a manifest contains a suspicious field, but whether the service account, namespace, network exposure, and runtime path combine into an exploitable control plane.

Container-to-cloud credential theft. TeamTNT analyses describe campaigns targeting exposed container or Kubernetes environments and attempting to retrieve AWS credentials through instance metadata or local credential material [16], [17]. This pattern maps directly to context-dependent IaC review because the dangerous path often crosses layers: a container exposure, node or pod identity, metadata access, IAM permissions, and downstream cloud resources. SecReviewAgent’s persistent architecture memory is designed to retain these relationships across reviews so repeated changes to identities, network paths, and data stores can be interpreted together.

TABLE IV
LATENCY BREAKDOWN (SECONDS)

Component	Cold	Warm	P50	P99
GitHub API fetch	0.8	0.8	0.6	2.1
Context load (S3)	–	0.04	0.03	0.12
Full repository scan	12.3	–	10.1	18.7
Terraform parsing	0.3	0.3	0.2	0.8
LLM analysis	4.1	3.2	3.0	8.4
Output formatting	0.2	0.2	0.1	0.4
GitHub API comment	0.7	0.7	0.5	1.8
Total	18.4	4.9	4.4	12.2

TABLE V
USER STUDY RESULTS

Metric	Without tool	With SecReviewAgent
Median review time	23.4 min	12.7 min
Mean findings per PR	1.8	2.4
Finding precision	0.72	0.91
Finding recall	0.61	0.82
Confidence (1–5)	3.1	4.2
Would use in production	–	89%

D. Latency and Cost

Persistent memory yields a substantial warm-path benefit by avoiding repeated repository scans. Table IV summarizes latency by component.

Warm reviews are therefore 73% faster than cold reviews. Cost is similarly modest: the blended average is reported as \$0.023 per review under an assumption that 15% of reviews incur cold-start-style repository initialization. The key deployment point is that most of the latency gain comes from removing repeated repository reconstruction rather than from changing model inference itself, which aligns directly with the persistent-memory design claim.

E. User Study

A controlled study with 42 security practitioners used a within-subjects design, where each participant reviewed two pull requests with the tool and two without it. Participants were recruited from cloud security, platform engineering, application security, and site reliability roles. Each participant reviewed matched pull requests of comparable size and risk category; task order was counterbalanced to reduce learning effects. After each task, participants recorded review time, findings, confidence, and whether they would use the workflow in production. Precision and recall were computed against the adjudicated labels from the corpus. Table V summarizes the results.

Participants reported that persistent context reduced repository spelunking and made findings easier to interpret when reviewing unfamiliar services. The main concerns were false negatives, verbosity on simple issues, and a need for tunable severity thresholds. Because the user study includes 42 participants with a within-subjects design and a small number of tasks per participant, we treat these outcomes as strong

directional evidence rather than a definitive human-factors claim.

VI. DISCUSSION

SecReviewAgent’s strongest contribution is architectural context retention. In this evaluation, persistent memory improves context-dependent security detection while also reducing latency on repeated reviews. The results suggest that IaC review quality is limited not only by rule coverage, but also by whether repository context is reused effectively. For financial-engineering organizations, this aligns with a practical control objective: increasing IaC delivery speed without weakening evidence quality for risk, compliance, and model-governance reviews.

An important interpretive point is that the strongest gains occur precisely where repository memory should matter most: cross-resource, context-dependent findings. This pattern is more informative than aggregate F1 alone because it aligns the empirical benefit with the proposed mechanism. Even so, the paper should not be read as claiming that persistent memory fully explains all gains; prompt design, parser quality, and output structure also contribute.

At the same time, several limitations remain. Repository context may become stale if infrastructure changes occur outside version control. Large repositories may exceed processing budgets even with hierarchical compression. Cross-repository architectures remain only partially modeled. Most importantly, the system does not eliminate the need for human review. Novel attacks, ambiguous business logic, or subtle environmental assumptions can still evade automated analysis.

From an evidence standpoint, the strongest support in this paper comes from the curated benchmark and the user study. The latency and cost measurements strengthen practical relevance, but they remain implementation-specific and should be interpreted with caution.

VII. THREATS TO VALIDITY

Internal validity is influenced by the quality of human labeling, prompt design, and baseline configuration choices. Dual labeling with third-reviewer resolution mitigates but does not remove annotation risk. The no-context LLM baseline is intended as an ablation-style comparison, but exact parity between commercial review settings is difficult to guarantee.

External validity is limited by the 23-repository corpus, even though it spans several domains and cloud providers. Construct validity is affected by the subjectivity of severity classification, although calibration against industry-style severity reasoning improves consistency. Finally, latency and cost measurements depend on the prototype deployment configuration and should be interpreted as feasibility evidence rather than general performance guarantees. We also did not preregister hypotheses or analysis plans for this retrospective study, which leaves room for unintentional analyst degrees of freedom.

VIII. RELATED WORK

Prior work on IaC security analysis has focused primarily on static misconfiguration detection, including Checkov, tfsec, KICS, and Terrascan. Empirical studies of IaC vulnerabilities have documented the prevalence of security smells and policy violations in declarative infrastructure code. In parallel, recent research on LLM-assisted code review and software security has demonstrated the promise of transformer-based reasoning for review assistance, vulnerability detection, and explanation generation.

SecReviewAgent differs from these lines of work in one key respect: it introduces persistent architecture memory into the review process. Rather than applying an LLM in a stateless fashion to each review, it accumulates repository knowledge and uses that knowledge to interpret future changes. This design is particularly natural for IaC because architectural relationships are first-class and often more important than single-line code changes.

IX. CONCLUSION

This paper presented SecReviewAgent, a context-aware IaC security review system that augments LLM reasoning with persistent architecture memory. Across 847 pull requests, the system achieved precision of 0.89, F1 of 0.83, and a $2.4\times$ improvement in context-dependent issue detection over a no-context LLM baseline. Warm reviews were 73% faster than cold reviews because repository context could be reused rather than rebuilt. A 42-participant user study further indicated substantial reductions in review time and improvements in finding quality.

In practical terms, the results suggest that durable context is one of the missing pieces in AI-assisted software security. For IaC review, a persistent architectural model helps bridge the gap between local diffs and system-level security meaning. The present results support SecReviewAgent as a useful systems prototype, while also leaving clear work for future ablation, larger multi-repository studies, and stronger longitudinal validation.

ACKNOWLEDGMENT

The authors thank the security practitioners who participated in the user study and the open-source community for providing test repositories.

REFERENCES

- [1] Bridgecrew, “Checkov: Policy-as-code for infrastructure,” <https://www.checkov.io/>, 2024.
- [2] Aqua Security, “tfsec: Security scanner for Terraform,” <https://github.com/aquasecurity/tfsec>, 2024.
- [3] Checkmarx, “KICS: Keeping Infrastructure as Code Secure,” <https://kics.io/>, 2024.
- [4] Tenable, “Terrascan: Detect compliance and security violations,” <https://runterrascan.io/>, 2024.
- [5] HashiCorp, “State of Cloud Strategy Survey,” 2024.

- [6] CNCF, “Cloud Native Survey 2023,” Cloud Native Computing Foundation, 2023.
- [7] Z. Li et al., “Automating Code Review Activities by Large-Scale Pre-training,” arXiv:2203.09095, 2022.
- [8] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyanyk, and G. Bavota, “Using Pre-Trained Models to Boost Code Review Automation,” in *Proc. IEEE/ACM International Conference on Software Engineering (ICSE)*, pp. 2291–2302, 2022.
- [9] S. Ducasse and D. Pollet, “Software Architecture Reconstruction: A Process-Oriented Taxonomy,” *IEEE Transactions on Software Engineering*, vol. 35, no. 4, pp. 573–591, 2009.
- [10] J. Garcia, D. Popescu, C. Mattmann, N. Medvidovic, and Y. Cai, “Enhancing Architectural Recovery Using Concerns,” in *Proc. IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 552–555, 2011.
- [11] A. Rahman, C. Parnin, and L. Williams, “The Seven Sins: Security Smells in Infrastructure as Code Scripts,” in *Proc. IEEE/ACM International Conference on Software Engineering (ICSE)*, pp. 164–175, 2019.
- [12] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, “Examining Zero-Shot Vulnerability Repair with Large Language Models,” in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2023.
- [13] Office of the Comptroller of the Currency, “OCC assesses \$80 million civil money penalty against Capital One,” News Release 2020-101, Aug. 2020.
- [14] U.S. Department of Justice, Western District of Washington, “United States v. Paige Thompson,” Case Summary, 2019.
- [15] RedLock CSI Team, “Defending against cryptojacking attacks in the cloud,” Cloud Security Trends Report, 2018.
- [16] Sysdig Threat Research Team, “Threat news: TeamTNT stealing credentials using EC2 instance metadata,” 2021.
- [17] CERT-EU, “A cryptomining worm that steals AWS credentials,” Threat Memo, 2020.

Disclaimer: The views and opinions expressed in this article are solely those of the authors and do not represent or reflect the views, positions, policies, or opinions of the authors’ employer or any affiliated organization. The content is provided for informational purposes only. The employer makes no representations or warranties regarding the accuracy or completeness of the content and does not endorse or accept responsibility for any statements, conclusions, or materials contained herein.